```
      System.out.print("Do you wish to i = insert, d = delete, ");
      System.out.print("p = print, or s = stop: ");
      response = scanner.next();
    }

    System.out.println();
    System.out.println("End of Program");
    System.out.println();

  }
}
```

6.5 Doubly Linked Lists

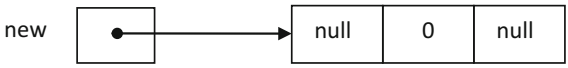
Doubly linked lists are like singly linked lists, except that in addition to a `next` reference they also have a `back` reference. The partial `Node` class for this type of list would appear as follows:

```
public class Node {
  private Node back;
  private int data;
  private Node next;

}
```

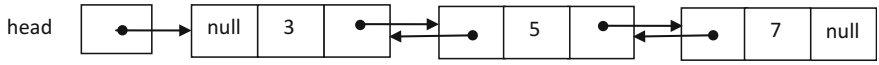
Or alternatively as follows:

```
public class Node {
  private int data;
  private Node back,next;

}
```

The order of `back` and `next` do not matter in memory, but the former one reflects more how a `Node` is drawn, and the second one reduces the number of keystrokes. Given the former, and the creation of a new node, the appearance would be as follows:

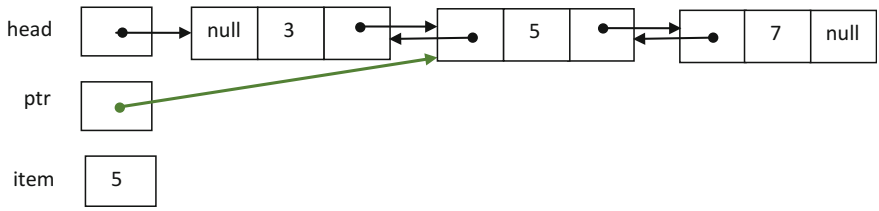


Further, the constructor in Fig. 5.1 would need to be altered to set `back` to `null` and both a `setBack` and `getBack` method would be needed to be added to the class. Assuming the existence of a doubly linked list, it could look as follows:



In looking at the doubly linked list above, what would be the advantage of a doubly linked list over a singly linked list? Given the existence of a `back` pointer for each node, it would be that a `prev` reference is not necessary and there could be fewer lines of code in the `insert` and `delete` methods. However, with every advantage there often seems to a corresponding disadvantage. What appears to be the disadvantage of a doubly linked list over a singly linked list? In looking at the diagram it should be apparent that doubly linked lists require more memory than singly linked lists.

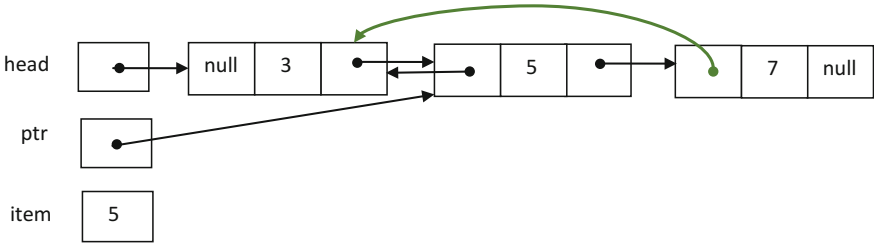
Reusing the preceding diagram to demonstrate how code might be written for a doubly linked list, assume that the number `5` is to be deleted. In this case `ptr` would refer to the node containing the `5` as shown below:



Given the drawing above, which references need to be altered? Only the `back` reference of the node containing the `7` needs to refer to the node containing the `3`, and the `next` reference of the node containing the `3` needs to reference the node with the `7`. Which reference should be altered first: the `next` reference of the node containing the `3`, or the `back` reference of the node containing the `7`? Although in this case either can be done first, one should always be careful not to delete a reference that might be needed later to complete the task at hand. In choosing one, the back reference of the node containing the `7` needs to refer to the node containing the `3`, so the following instruction can accomplish this task:

```
ptr.getNext().setBack(ptr.getBack());
```

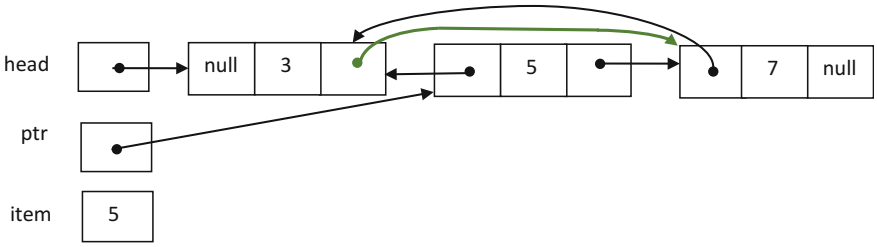
Through `ptr`, the above line of code first gets from `next`, in the node containing the `5`, the reference to the node containing the `7`. It then sets the `back` reference in the node containing the `7` to the same as the `back` reference of the node containing the 5 which is referencing the node containing the `3` as follows:



Similarly, the following instruction causes the `next` section of the node containing the `3` to reference the node containing the `7`:

```
ptr.getBack().setNext(ptr.getNext());
```

Again through `ptr`, it first gets from `back`, in the node containing the `5`, the reference to the node containing the `3`. It then sets the `next` reference in the node containing the 3 to the same as the `next` reference of the node containing the 5 which is referencing the node containing the `7` as follows:



Do the `next` and `back` references for the node with the `5` need to be altered? No, because when `ptr` is `null`, the garbage collector will return the node with the `5` along with its two references back to the heap.

Given that each line of code might be more complex than with singly linked nodes, not having to deal with a `prev` reference makes for fewer lines of code. Further, even though only two lines of code are provided in the section for deleting from the middle of the list, the exercises at the end of this chapter provide further practice with doubly linked lists.

Although doubly linked lists provide an alternative way to handle lists, the use of two references in a node foreshadow the construction of more sophisticated data structures called binary trees as will be discussed in Chap. 8.

6.6 The Node Class as an Inner Class

The LinkedList class discussed in previous sections depends on an external Node class. However, when dealing with linked lists, there is no reason to have access to an individual node in the list from the main program. The result is that the Node class can be made to be a private inner class in the LinkedList class, so that linked list structures are contained in themselves. This would eliminate the need for accessor and mutator methods in the Node class, and programs introduced in later chapters implemented with nodes, such as binary trees, can also be simpler compared to ones using the external Node class.

The following Node class without an accessor or mutator can be added to the LinkedList class which will be renamed to LinkedListInnerNode.

```
private class Node {

  private int data;
  private Node next;

  public Node() {
    this(0);
  }
  public Node(int data) {
    this.data = data;
    next = null;
  }
}
```

When the Node class is an inner class, the LinkedListInnerNode class has direct access to the private data members of the Node class, data and next, without using public accessor and mutator methods. Therefore, the statements that include get methods such as ptr.getData() and ptr.getNext() can be replaced by ptr.data and ptr.next respectively using dot notation. And newp.setData(item) and newp.setNext(ptr) are changed to newp.data = item and newp.next = ptr. The complete LinkedListInnerNode class with the inner Node class is shown below:

```
public class LinkedListInnerNode {

  private class Node {

    private int data;
    private Node next;

    public Node() {
      this(0);
    }

    public Node(int data) {
      this.data = data;
      next = null;
    }
  }

  private Node head;

  public LinkedListInnerNode(){
    head = null;
  }

  public boolean insert(int item) {
    boolean inserted;
    Node ptr,prev;
    inserted = false;
    ptr = head;
    prev = null;
    while(ptr ! = null && ptr.data < item){
      prev = ptr;
      ptr = ptr.next;
    }
    if (ptr == null || ptr.data == item) {
      inserted = true;
      Node newp = new Node();
      newp.data = item;
      newp.next = ptr;
      if (prev == null)
        head = newp;
      else
        prev.next = newp;
    }
    return inserted;
  }
```

```
  public boolean delete(int item) {
    boolean deleted = false;
    Node ptr, prev;
    //Body of method
    return deleted;
  }

  public void printRecursive() {
    System.out.print("List Recursive: ");
    printR(head);
    System.out.println();
  }

  private void printR(Node p){
    if(p ! = null) {
      System.out.print(data + " ");
      printR(p.next);
    }
  }
}
```

The only change that needs to be made in the main program, which uses the `LinkedList` class, is to replace the declaration and creation of a linked list,

```
LinkedList list;
list = new LinkedList();
```

with the following code:

```
LinkedListInnerNode list;
list = new LinkedListInnerNode();
```

6.7 Complete Programs: List of User Defined Objects

In this section, an application which keeps track of students' information is considered. What is the best way to implement this application? Some of the common operations applied on the data include addition, deletion, and search. Keeping the data in an organized fashion is good to help facilitate retrieval. Therefore, the

answer to the question is to keep the information in an ordered list. As discussed before, a list can be implemented using arrays or linked lists. Here a linked list is used. Since the items stored in the list are not simple primitive data types, a `LinkedList` class which accepts generic types is developed so that the list can store any type of items. The class that defines the node containing an object could be a separate class from the `LinkedList` class, which will be described in Sect. 6.7.1, or an inner class of the `LinkedList` class discussed in Sect. 6.7.2.

6.7.1 Generic `LinkedList` Class with External `NodeGeneric` Class

To define linked lists which keep any user defined objects, the type of data in the node of the list should be a generic type so that a new `LinkedList` class does not have to be written every time items of different types are stored. The `NodeGeneric<T>` class has already been implemented in Fig. 5.6, which will be used with a generic linked list to be developed next.

As discussed in Sect. 4.5, the type `int` in the `LinkedList` class shown in Fig. 6.3 will be replaced by the type parameter `T`. The reference type `Node` will be changed to the generic type `NodeGeneric<T>`. Similar to the `ListArrayGeneric` class, the `LinkedListGeneric` class needs to compare items to determine where the new item should be inserted in the list, and the comparison `ptr.getData() < item` in Fig. 6.3 does not work.

In the generic class, this will compare the references to the objects, not the contents. Just as in the `ListArrayGeneric` class, the `compareTo` method that is included in the `Comparable` interface is used to compare two objects. How to compare two objects differs from one object to an other, therefore actual implementation of the `compareTo` method would be in the class that defines the objects. Also, since the condition `ptr.getData() ! = item` in Fig. 6.3 does not work with objects, the `equals` method will be implemented. The complete `LinkedListGeneric` class which uses `NodeGeneric<T>` class is shown in Fig. 6.4, and the definitions of the methods `compareTo` and `equals` are discussed next.

Since the items stored in the list for this application consist of student information such as name, ID number, and major, a class which defines student objects will be implemented. There will be three data members describing a student: `name` and `major` of type `String`, and `id` of type `int`. For each data member, an accessor and a mutator are included. Because the class will implement the `compareTo` method which is included in the `Comparable` interface, the class will be named as `StudentComparable`. Because students are ordered by their `id` the `compareTo` method compares the `id` number of two students, and returns an integer. Specifically, it returns a positive integer, zero, or a negative integer if the id

```
public class LinkedListGeneric<T extends Comparable<T>> {

   private NodeGeneric<T> head;

   public LinkedListGeneric() {
      head = null;
   }

   public boolean insert(T item) {
      boolean inserted;
      NodeGeneric<T> ptr,prev;
      inserted = false;
      ptr = head;
      prev = null;
      while(ptr != null && ptr.getData().compareTo(item) < 0) {
         prev = ptr;
         ptr = ptr.getNext();
      }
      if(ptr == null || !(ptr.getData().equals(item))) {
         inserted = true;
         NodeGeneric<T> newp = new NodeGeneric();
         newp.setData(item);
         newp.setNext(ptr);
         if (prev == null)
            head = newp;
         else
            prev.setNext(newp);
      }
      return inserted;
   }
   public boolean delete(int item) {
      boolean deleted=false;
      Node ptr, prev;
      // Body of method
      return deleted;
   }

   public void printRecursive() {
      System.out.print("List Recursive: ");
      printR(head);
      System.out.println();
   }

   private void printR(Node p){
      if(p != null) {
         System.out.print(p.getData()+" ");
         printR(p.getNext());
      }
   }
}
```

Fig. 6.4 `LinkedListGeneric` class

of the object that is calling the method is greater than, equal to, or less than the one of the specified object, respectively. This method will be used in the `insert` and `delete` methods. The code below is the `compareTo` method:

```
public int compareTo(StudentComparable otherStudent) {
   int result;
   if(id < otherStudent.getId())
     result = -1;
   else
    if(id > otherStudent.getId())
      result = 1;
    else
      result = 0;
   return result;
}
```

The `equals` method, which can be used in the `insert` and `delete` methods, is also implemented in the `StudentComparable` class. The `equals` method takes a reference to the object of type `Object` which means an object of any type to maintain the generic nature of the code. The object will be typecast into the `StudentComparable` type so that the `id` number of the object calling the method can be compared to the `id` of the object passed to the method as a parameter. As discussed in Chap. 2, the class `Object` is the root of the class hierarchy in Java, meaning the `Object` class is directly or indirectly a superclass of all the user defined and predefined classes. The definition of the `equals` method is shown below:

```
public boolean equals(Object otherStudent) {
  StudentComparable otherStudentObject
         = (StudentComparable)otherStudent;
  return (id == otherStudentObject.id);
}
```

Lastly, the `toString` method shown below is added at the end to return a string representation of the contents of the data members of an object. The method would return the `name`, `id`, and `major` of the student and would be written as follows:

```
  public String toString() {
    return ("Student named " + name + " with id " + id +
         " and major " + major);
  }
```

Putting everything discussed above together, the complete `StudentComparable` class is shown below:

```
public class StudentComparable implements Comparable<StudentComparable> {

  private String name, major;
  private int id;

  public StudentComparable(String name, String major, int id) {
    this.name = name;
    this.major = major;
    this.id = id;
  }

  public String getName() {
    return name;
  }

  public String getMajor() {
    return major;
  }



  public int getId() {
    return id;
  }

  public void setName(String name) {
    this.name = name;
  }

  public void setMajor(String major) {
    this.major = major;
  }

  public void setId(int id) {
    this.id = id;
  }
```

```
  public int compareTo(StudentComparable otherStudent) {
     int result;
     if(id < otherStudent.getId())
       result = -1;
     else
      if(id > otherStudent.getId())
        result = 1;
      else
        result = 0;
     return result;
  }

  public boolean equals(Object otherStudent) {
    StudentComparable otherStudentObject
                = (StudentComparable)otherStudent;
    return (id == otherStudentObject.id);
  }

  public String toString() {
    return ("Student named " + name + " with id " + id +
         " and major " + major);
  }
}
```

Now it is time to test three classes discussed above. A main program which creates an ordered linked list with 3 students is shown below.

Sample output from the above program is:

```
List:
Student named George with id 1212 and major CS
Student named Maya with id 1234 and major MIS
Student named Joffrey with id 3456 and major CS
```

where students are sorted by their `id` number.

6.7.2 Generic `LinkedList` Class with Internal `NodeGeneric` Class

In this section, a linked list with an inner node class is used to perform the same task discussed in the previous section. More specifically, the `NodeGeneric<T>` class in Fig. 5.6 without accessors and mutators is included in the `LinkedListGeneric<T>` class in Fig. 6.4. And it will be called the `LinkedListGenericInnerNode` class as shown below.

```
public class LinkedListGenericInnerNode<T extends Comparable<T>> {

  private class NodeGeneric<T>{

    private T  data;
    private NodeGeneric<T> next;

    public NodeGeneric() {
      this(null);
    }

    public NodeGeneric(T data) {
      this.data = data;
      next = null;
    }
  }

  private NodeGeneric<T>head;

  public LinkedListGenericInnerNode(){
    head = null;
  }



  public boolean insert(T item) {
    boolean inserted;
    NodeGeneric<T>ptr,prev;
    inserted = false;
    ptr = head;
    prev = null;
    while(ptr ! = null && ptr.data.compareTo(item) < 0){
      prev = ptr;
      ptr = ptr.next;
    }
    if(ptr == null || !(ptr.data.equals(item))) {
      inserted = true;
      NodeGeneric<T>newp = new NodeGeneric();
      newp.data = item;
      newp.next = ptr;
      if(prev == null)
        head = newp;
      else
        prev.next = newp;
    }
```

```
    return inserted;
  }

  public boolean delete(T item) {
    boolean deleted;
    NodeGeneric<T>ptr, prev;
    return deleted;
    //Body of method
  }

  public void printRecursive() {
    System.out.print("List Recursive: ");
    printR(head);
    System.out.println();
  }

  private void printR(NodeGeneric<T> p){
    if(p ! = null) {
      System.out.print(p.data + " ");
      printR(p.next);
    }
  }
}
```

The `studentComparable` class discussed in the previous section can be used here. The only change that needs to be made in the main program in Fig. 6.5 is to modify the declaration and creation of a linked list. Their type is now `LinkedListGenericInnerNode` instead of `LinkedListGeneric` shown below:

```
public class TestLinkedListGenericStudentComparable {

  public static void main(String[] args) {

    LinkedListGenericInnerNode<StudentComparable>list
        = new LinkedListGenericInnerNode<StudentComparable>();
    LinkedListGenericInnerNode<StudentComparable>ptr;

    StudentComparable student1
        = new StudentComparable("George", "MIS", 1234);
    StudentComparable student2
        = new StudentComparable("Maya", "CS", 1212);
    StudentComparable student3
        = new StudentComparable("Joffrey", "CS", 3456);
```

```
    list.insert(student1);
    list.insert(student2);
    list.insert(student3);

    list.printRecursive();
  }
}
```

6.8 Summary

- Drawing pictures can be extremely helpful when trying to write or debug code for linked lists.
- Ordered linked lists have each node in a prescribed order such as ascending order for integers and alphabetical order for strings.
- It is often helpful to write code for the generic case first and then proceed to write code for the other cases, such as inserting or deleting from the beginning or end of a list.
- Doubly linked lists have the advantage that they do not need an extra variable referencing the previous node in the list, but the disadvantage is that each node takes up more memory referencing the previous node.

```
public class TestLinkedListGenericStudentComparable {

   public static void main(String[] args) {

      LinkedListGeneric<StudentComparable> list
            = new LinkedListGeneric<StudentComparable>();
      LinkedListGeneric<StudentComparable> ptr;

      StudentComparable student1
            = new StudentComparable("George", "MIS", 1234);
      StudentComparable student2
            = new StudentComparable("Maya", "CS", 1212);
      StudentComparable student3
             = new StudentComparable("Joffrey", "CS", 3456);

      list.insert(student1);
      list.insert(student2);
      list.insert(student3);

      list.printRecursive();
   }
}
```

Fig. 6.5 Main program using the generic `LinkedList` class

- Doubly linked lists require fewer lines of code, but each line of code tends to be more complicated.
- The use of inner class can eliminate the need for accessor and mutator methods, especially when dealing with more complex data structures.

6.9 Exercises (Items Marked with an * Have Solutions in Appendix B)

*1. Assuming the list `intList` of type `LinkedList` in Fig. 6.3 is initially empty, draw a list after walking though the following code segment:

```
intList.insert(41);
intList.insert(6);
intList.delete(5);
intList.delete(6);
intList.insert(78);
intList.insert(5);
intList.insert(79);
intList.delete(41);
intList.insert(5);
```

2. Assuming a list `characterList` of type `LinkedListGeneric<Character>` which is an object of the generic class `LinkedListGeneric<T>` in Fig. 6.4 is initially empty, draw a list after walking through the following code segment:

```
characterList.insert('U');
characterList.insert('V');
characterList.delete('U');
characterList.insert('W');
characterList.delete('A');
characterList.insert('P');
characterList.insert('Q');
characterList.delete('P');
characterList.insert('C');
```

3. Write a method `addToTail` that can be added to the `LinkedList` class in Fig. 6.3 which adds a node at the end of the list. Assume that the existence of a variable `tail` which reference the last node in a linked list.
*4. Write a method `deleteTail` that can be added to the `LinkedList` class in Fig. 6.3 which removes the last node from the list. The method does not accept anything and returns `true` if the list contained at least one node and `false` if the list was empty.
5. Write a method `delete` discussed in Sect. 6.2 that can be added to the `LinkedList` class in Fig. 6.3.
6. Write a method `size` that can be added to the `LinkedList` class in Fig. 6.3 which returns the number of nodes in the list. What other changes need to be made to the other methods in order to make this method work better?
*7. Write a method `isEmpty` that can be added to the `LinkedList` class in Fig. 6.3 which returns `true` if the list is empty.
8. Write a method `clear` that can be added to the `LinkedList` class in Fig. 6.3 which clears an exisiting list by making it an empty list.
9. Develop a class `LinkedListDuplicate` derived from the `LinkedList` class in Fig. 6.3 which accepts duplicates. The new class should contain all the methods in the `LinkedList` class. The `delete` method should delete all occurrences of an item from the list.
10. Draw a list after the complete program in Fig. 6.5 is executed.
11. Rewrite the `LinkedList` class in Fig. 6.3 for a doubly linked list.
12. Write a method `copy` that can be added to the class implemented for question 11. The method accepts a doubly linked list and makes a duplicate of it.
13. Write a program to find a difference of two given sets of integers stored in a linked list using `LinkedListGeneric<T>` in Fig. 6.4. For example, if two integer sets are `P = {4, 7, 1, 10}` and `Q = {4, 2, 5, 1, 3}`, the differences of these two sets are `P - Q = {7, 10}` and `Q - P = {2, 5, 3}`.
14. Implement an unordered LinkList class.